

2024 Sept OPPE 1 - SET 2

Section 1 - Problem 1

Problem Statement

Problem statement:

- You need to write a function **is_positive_odd_or_negative_even** ,
- that checks if an integer is:
 - **a positive odd number**
 - OR**
 - **a negative even number**
- **Return True** if the number meets one of these conditions; otherwise, **return False**.
- This is a **function type question**, you don't have to take input or print the output, just have to complete the required function definition.

Example and Explanation:

- Let our input be **a = 4**. Our argument here is an integer.
- We need to check if 4 is
 - either positive **AND** an odd number
 - OR**
 - negative **AND** an even number

In our case,

4 is divisible by 2 so it is **even**.

BUT

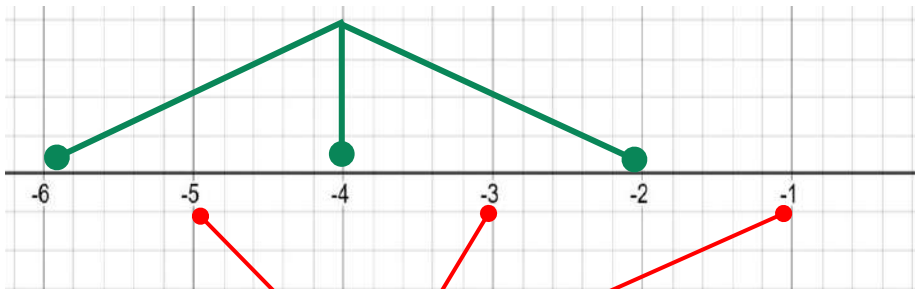
4 is not a **negative number** so the condition is not satisfied. So, **our output should be False**.

Logic Visualization on a number line

Negative numbers : LEFT side

Left side and Even = TRUE

-6, -4, and -2 are divisible by 2. They are negative and even.



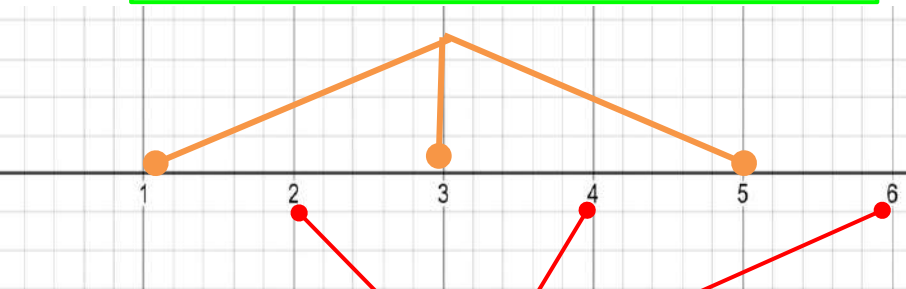
-5, -3, and -1 are not divisible by 2. They are negative and odd.

Left side and Odd = FALSE

Positive numbers : RIGHT side

Right side and Odd = TRUE

1, 3, and 5 are not divisible by 2. They are positive and odd.

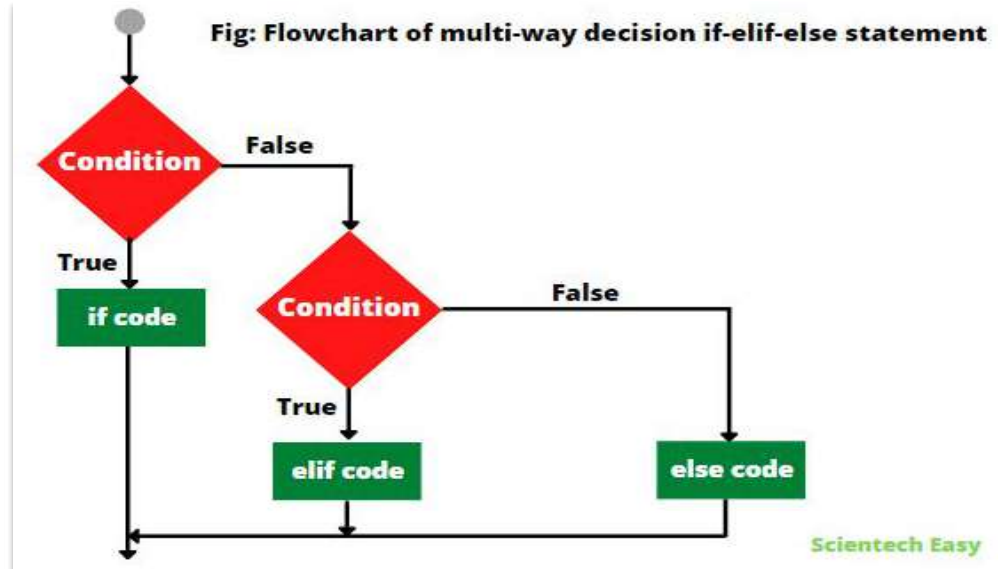


+2, +4, and +6 are divisible by 2. They are positive and even.

Right side and Even = FALSE

Concept of if - elif & else statements

- **if**, **elif**, and **else** help programs make decisions, like choosing different paths based on conditions.
- **if** checks the first condition, **elif** checks other conditions if needed, and **else** runs when none are true.
- They work like a decision tree, where only one matching block executes from top to bottom. You **cannot have an else statement without an if statement but the reverse is possible**.

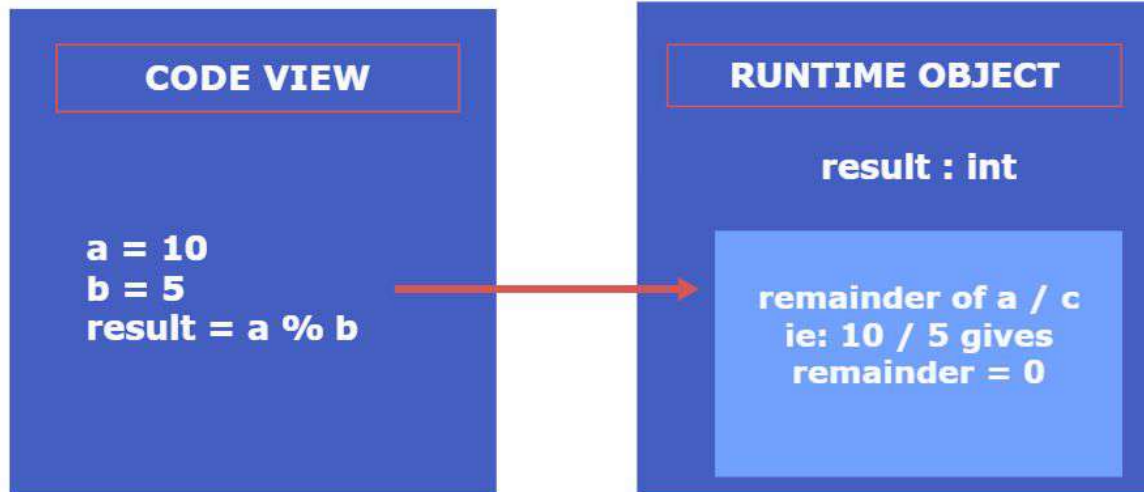


Concept of Modulo Operator

In Python, we use the modulo operator to **compute remainder** when two numbers are divided. **Modulo operator is: %**

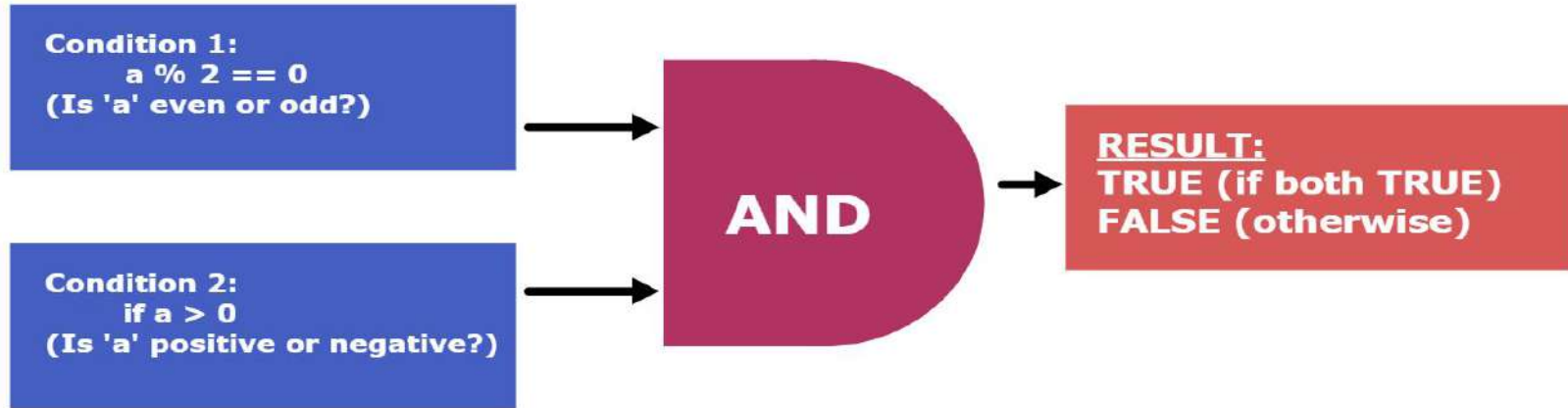
- $X \% Y$ returns the remainder of $X \div Y$
- If $X \% Y == 0$, then X is divisible by Y

Example: $10 \% 5 = 0$ (10 is divisible by 5)



Concept of Logical “AND” Operator

The Logical **and** operator returns **True** only if all the conditions mentioned are true.



CASE 1

is_positive_odd_or_negative_even(**15**)

As, $15 \% 2 \neq 0$ AND $15 > 0$
15 is Odd and Positive.
Result (TRUE AND TRUE) is **TRUE**

CASE 2

is_positive_odd_or_negative_even(**-3**)

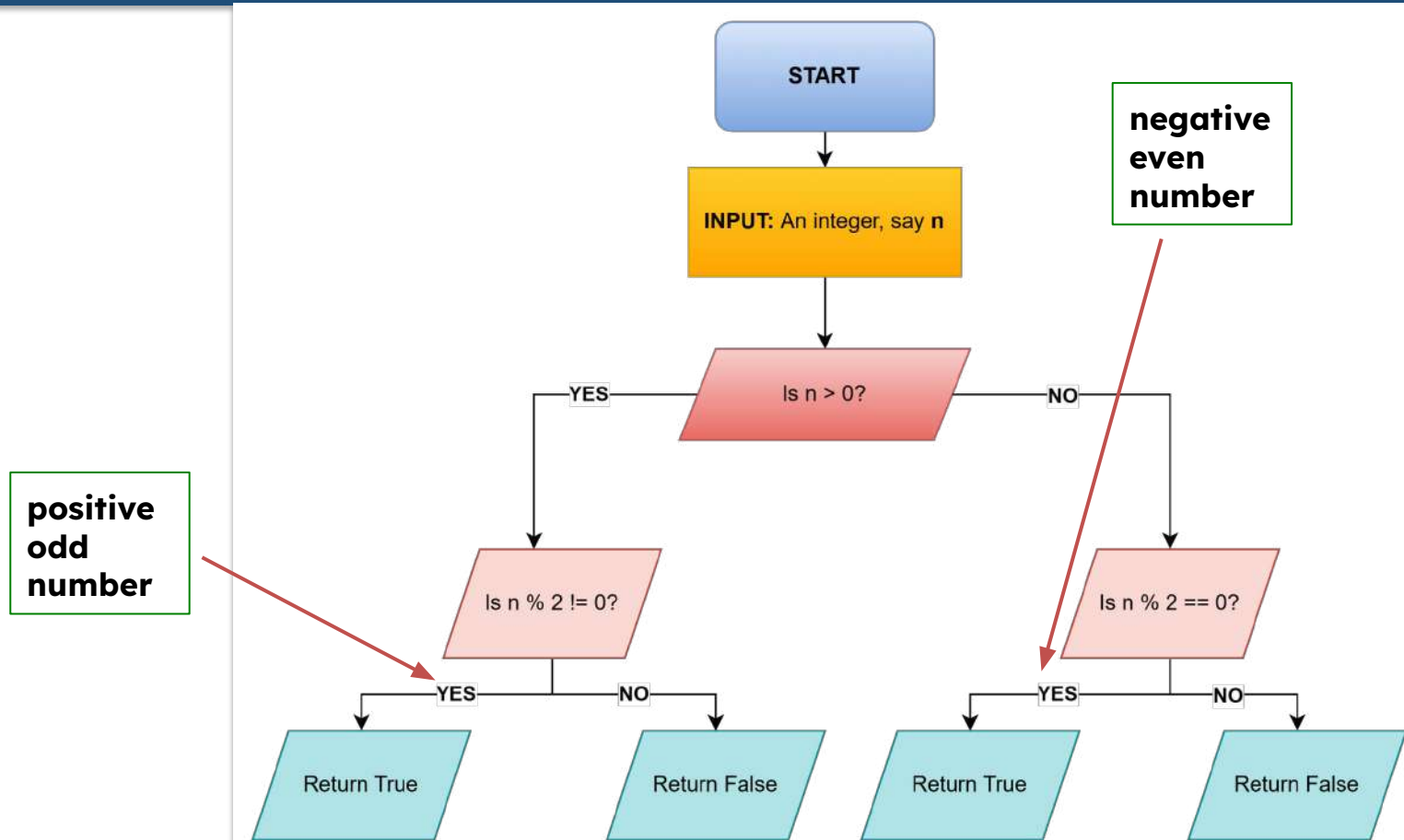
As, $-3 \% 2 \neq 0$ AND $-3 < 0$
-3 is Odd and Negative
Result (FALSE AND TRUE) is **FALSE**

CASE 3

is_positive_odd_or_negative_even(**-8**)

As, $-8 \% 2 == 0$ AND $-8 < 0$
-8 is Even and Negative.
Result (TRUE AND TRUE) is **TRUE**

Logic Flowchart



Our Solution Approach

THE BLACK BOX

**INPUT
(1 INTEGER)**

1: Check if number is divisible by 2 AND lesser than 0, then return TRUE

Use- if $a \% 2 == 0$ and $a < 0$

2: Check if number is not divisible by 2 and greater than 0, then return TRUE

Use- elif $a \% 2 != 0$ and $a > 0$

3: Anything otherwise goes in else condition, return False. (eg. 0)

Use- else:

**OUTPUT
(BOOLEAN)**

Solution 1: The Explanatory Approach

```
def is_positive_odd_or_negative_even(a: int) -> bool:
```

```
    """
```

```
    Checks if the integer `a` is either (positive AND odd) OR  
(negative AND even)
```

```
    """
```

```
    # Usage of if - elif - else statement:
```

```
    if a % 2 == 0 and a < 0:
```

```
        return True
```

```
    elif a % 2 != 0 and a > 0:
```

```
        return True
```

```
    else:
```

```
        return False
```

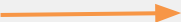
Function
definition with
integer **a** as
parameter

Use **if - else** conditional statement to
check if conditions hold, use
mathematical operators such as
modulo operator % and comparison
operator such as **greater than >**

Return **final result**:
True if **both** conditions
hold, or **else** return **False**

A step by step implementation of solution for better understanding

Solution 2: An alternate way using 'nested if's'

```
def is_positive_odd_or_negative_even(n):  
    if n > 0:  
        if n % 2 != 0:  
            return True  
    if n < 0:  
        if n % 2 == 0:   
            return True  
    return False
```

Usage of nested if statements:

- Nested if statements allow checking a condition inside another condition
- The inner if runs only if the outer if condition is True
- This helps break complex logic into smaller, step-by-step checks

An alternative solution using nested if statements instead of elif and else

Solution 2 - The Concise Approach

```
def is_positive_odd_or_negative_even(n):  
    return (n > 0 and n % 2 != 0) or (n < 0 and n % 2 == 0)
```



- This direct boolean one liner way checks the condition using logical operators **in one statement instead of multiple if** blocks.
- It first checks if the number is greater than 0 and odd, and also checks if it is less than 0 and even.
- The **and** ensures both parts of a condition are true, and the **or** allows either valid case to return **True**.
- If neither condition is satisfied, it automatically returns **False**.

The Pythonic Optimization: One-liner solution using logical AND operator.
Result: Identical output, neater code

Solution comparison

Step by Step Approach



- **Higher Readability**
- **Clearer understanding without any missing steps**
- **Best for beginners and debugging**
- **A little time consuming**
- **Best Usage: When problem statement itself is complicated**

Concise One - Liner Approach



- **Neat and shorter code**
- **Fast and time efficient**
- **Best for competitive programming/ when coding with time constraints**
- **Fewer memory assignments**
- **Might get confusing as problem statement gets challenging**
- **Best Usage: for simpler tasks to save time**

Key Takeaways

- 🔑 **and** is used to combine two conditions that must both be true (e.g., positive and odd).
- 🔑 **or** allows either of two valid cases to return **True**.
- 🔑 The modulo operator **%** checks odd/even ($n \% 2 == 0 \rightarrow \text{even}$, $n \% 2 != 0 \rightarrow \text{odd}$).
- 🔑 Comparison operators like **>** and **<** determine whether the number is positive or negative.
- 🔑 Logical operators help write clear, compact decision logic in one expression.
- 🔑 **if** checks a condition and runs code only when the condition is **True**
- 🔑 **elif** allows checking another condition if the first one is **False**
- 🔑 **else** runs when none of the conditions are **True**, acting as the default case.